

ifcc — Mini-compileur C

Sous-ensemble de C → assembleur x86-64, écrit en C++ / ANTLR4

Projet PLD-COMP — INSA Lyon

Luka Maret · Valentin Dury · Sarah Ripoche · Adina Valkova · Louis Zyzelewicz · Clément Dubois

Ce qu'on a construit

Un vrai pipeline de compilation, en **3 passes** séparées :

```
source .c → Lexer/Parser ANTLR → arbre
          → SymbolTableVisitor   (passe 1 : sémantique)
          → IRGeneratorVisitor   (passe 2 : IR = graphe de flot)
          → X86Backend           (passe 3 : assembleur)
          → x86-64 AT&T sur stdout
```

Critère de réussite **différentiel** : un test passe si `ifcc` et **GCC** sont d'accord (même code de sortie, ou tous deux rejettent le programme).

122 cas de test automatisés (`tests/ifcc-test.py`).

Le langage supporté

- **Types** : `int` , `char` , `void` , pointeurs (`int*` , `int**`).
- **Arithmétique** : `+` `-` `*` `/` `%` , unaire `-` , précedence et parenthèses.
- **Bit-à-bit** : `&` `|` `^` . **Comparaisons** : `<` `>` `<=` `>=` `==` `!=` .
- **Variables** : déclarations multiples, init, affectations chaînées.
- **Contrôle de flux** : `if` / `else` / `else if` , `while` (imbriqués).
- **Fonctions** : définition, paramètres (>6 ⇒ pile), récursion, `putchar` / `getchar` .
- **Pointeurs** : `&x` , `*p` , `**pp` , écriture `*p = ...` .
- **Tableaux** : déclaration, init `{...}` (partielle), lecture/écriture indexée.

Passe 1 — Analyse sémantique

`SymbolTableVisitor` construit les tables **avant** toute génération de code :

- une table `map<string, Variable>` **par fonction**, + une table des fonctions (type de retour + types des paramètres) ;
- erreurs détectées : redéfinition, variable non déclarée, `void`, mauvais nombre d'arguments, fonction non définie, absence de `main`.

```
int main() { int a; int a; } // Error: 'a' is already declared
int main() { return f(1, 2); } // Error: mauvais nombre d'arguments
```

Passe 2 — L'IR : un graphe de flot de contrôle

```
ControlFlowGraph -owns→ [Block, Block, ...]  
Block            -owns→ [Instruction, ...] + arêtes true/false
```

- Chaque `visitXxx` renvoie le nom de la variable contenant son résultat
→ composition naturelle des expressions.
- Temporaires `$temp0, $temp1...` (`CFG::addTempVariable`), `$return` pour le retour.
- **21 classes d'instructions** : `Add, Subtract, Copy, LoadConstant, Negate, Bitwise*`, comparaisons, `CallFunction, Reference, DereferenceRead/Write...`

Passe 2 — Optimisations à la volée

Pliage de constantes — les **opérations** entre constantes connues sont évaluées à la compilation : aucun `imull` / `addl` émis, juste des chargements.

```
int x = 2 + 3 * 4;    // 3*4 et 2+12 pliés → on charge directement 14
```

Simplifications algébriques — neutres / absorbants :

```
x + 0, x * 1 → x      x * 0, x & 0 → 0
```

Code mort — après `return`, bloc inatteignable détecté et signalé :

```
warning: 1 unreachable code block(s) detected
```

Passe 3 — Backend & double-dispatch

Le backend ignore tout de l'IR, l'IR ignore tout de l'assembleur :

```
instruction->generate(backend, output);           // Block::generateASM
void Add::generate(Backend &b, ostream &o) { b.emit(this, o); } // dispatch
// → résolution sur le type concret : X86Backend::emit(Add*)
```

→ Ajouter une **cible** = une nouvelle classe Backend (un ARMBackend existe).

Conventions **System V** : 6 premiers args en registres (%edi, %esi...), pile au-delà,
cadre aligné sur 16 octets, épilogue leave ; ret .

Exemple : une instruction émise

Toutes les opérandes vivent sur la pile → on **charge**, **calcule**, **range** :

```
void X86Backend::emit(Add *instr, ostream &o) {  
    o << "    movl " << varToLocation(instr->getLeft(), cfg) << ", %eax\n";  
    o << "    addl " << varToLocation(instr->getRight(), cfg) << ", %eax\n";  
    o << "    movl %eax, " << varToLocation(instr->getDestination(), cfg) << "\n";  
}
```

Comparaison `a < b` → `cmpl ; setl %al ; movzbl %al, %eax` (résultat 0 ou 1).

Tableaux (1/2) — l'idée

Pas de nouvelle instruction IR : un tableau est une **zone contiguë** réutilisant

`DereferenceRead` / `DereferenceWrite`, avec un **index** transmis au backend.

```
int arr[5] = {64, 34, 25, 12, 22}; // init partielle légale (reste à 0)
int x = arr[i];                    // lecture indexée
arr[j + 1] = arr[j];              // écriture indexée
```

```
// visitTable_expression_read_value : a[i]
std::string indexVar = asString(visit(ctx->expression()));
block->addInstruction(
    new DereferenceRead(block, elementType, tmp, tableName, 0, indexVar));
```

Tableaux (2/2) — l'assembleur

Un tableau fournit son adresse (`leaq`), un pointeur fournit son contenu (`movq`). Puis adressage indexé (base, index, scale) :

```
if (cfg->getVar(src).pointerDepth > 0)
    o << "    movq " << loc(src) << ", %rax\n";    // pointeur
else
    o << "    leaq " << loc(src) << ", %rax\n";    // tableau (zone contiguë)

o << "    movslq " << loc(index) << ", %rcx\n";    // index 32→64 bits
o << "    movl (%rax,%rcx," << size << "), %eax\n"; // base + index*scale
```

Exemple bout-en-bout — pliage

```
int main() { int x = 2 + 3 * 4; return x; }
```

Le pliage supprime les **opérations** (`imull`, `addl`); chaque constante reste un `LoadConstant`. Le `14` est calculé à la compilation :

```
.globl main
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $32, %rsp
main_entry:
    movl $0, -4(%rbp)
    movl $2, -8(%rbp)
    movl $3, -12(%rbp)
    movl $4, -16(%rbp)
    movl $12, -20(%rbp)
    movl $14, -24(%rbp)
    movl -24(%rbp), %eax
    movl %eax, -28(%rbp)
    movl -28(%rbp), %eax
    movl %eax, -4(%rbp)
    jmp main_exit
main_dead0:
main_exit:
    movl -4(%rbp), %eax
    leave
    ret
```

Bilan & points de conception

- **Séparation stricte** front-end / IR (CFG) / back-end.
- **Double-dispatch** pour découpler instructions et cibles.
- Optimisations réelles : **pliage**, simplifications, **code mort**.
- Tableaux intégrés **sans nouvelle instruction** (réutilisation + adressage indexé).
- **122 tests** différentiels vs GCC · build multi-plateforme (Makefile + Docker).